

Classi Wrapper

I tipi primitivi come oggetti

Package java.lang

Tipi primitivi e classi “wrapper”

- In varie situazioni, può essere comodo poter *trattare i tipi primitivi come oggetti*
 - per passarli per riferimento a una funzione
 - quando una funzione pretende come parametro (o restituisce) un **Object** (vedi Collection, List, ...)
- Una *classe “wrapper”* incapsula una variabile di un tipo primitivo
- La classe wrapper ha nome (quasi) identico al tipo primitivo che incapsula, ma con l’iniziale maiuscola.

Tipi primitivi e classi “wrapper”

Primitivi

boolean

char

byte

short

int

long

float

double

Wrapper

Boolean

Character

Byte

Short

Integer

Long

Float

Double

Tipi primitivi e classi “wrapper”

- Ogni classe wrapper definisce metodi per estrarre il valore della variabile incapsulata e viceversa.

Per estrarre il valore incapsulato:

Integer fornisce il metodo `intValue()`

Double fornisce il metodo `doubleValue()`

Boolean fornisce il metodo `booleanValue()`

Character fornisce il metodo `charValue()`

Tipi primitivi e classi “wrapper”

Per creare un oggetto da un valore primitivo:

`Integer i = new Integer(valore int)`

`Double d = new Double(valore double)`

`Boolean b = new Boolean(valore boolean)`

`Character c = new Character(valore char)`

Classi wrapper - esempio

```
public class EsempioWrapper {  
    public static void main(String  
args[]) {  
        int x = 35;  
        Integer ix = new Integer(x);  
        x = 2*ix.intValue();  
        System.out.println("ix =" +  
ix);  
        System.out.println("x =" + x);  
    }  
}
```

Estrae da un oggetto `Integer` il valore `int` incapsulato al suo interno, mediante `intValue()`

Output:
`ix = 35`
`x = 70`

Tipi primitivi e classi “wrapper”

- La classe wrapper contiene anche *funzioni che operano sul tipo primitivo corrispondente*
 - in particolare, conversione stringa / numero ...
 - ... e viceversa
- Attenzione: le funzioni che operano sul tipo primitivo sono *funzioni statiche, di classe*
 - infatti, non esiste alcun oggetto su cui invocare metodi
- Quindi, varie funzionalità sono disponibili in *doppia versione*:
 - funzione statica (per variabili di tipo primitivo)
 - metodi (per oggetti che siano istanze del tipo wrapper)

Tipi primitivi e classi “wrapper”

- Esempio: *toString()* nella classe *Integer*

- versione statica:

```
public static String toString(int x);
```

- prende un valore `int` e ne produce la rappresentazione sotto forma di stringa

- versione metodo:

```
public String toString();
```

- è implicitamente invocato su un oggetto `Integer`

- ne recupera il valore e ne produce la rappresentazione sotto forma di stringa.

Le classi wrapper

- Java distingue fra tipi primitivi e oggetti
 - I tipi primitivi come `int`, `double`, sono compatti e supportano gli operatori aritmetici
 - Le classi (`Integer`, `Double`) hanno più metodi:
`Integer.toString()`
 - Noi abbiamo bisogno di un “wrapper” per utilizzare i metodi:
`Integer ii = new Integer( i );`
`ii.hashCode()`
 - Allo stesso modo noi abbiamo bisogno di un tipo “unwrap” per utilizzare gli operatori aritmetici
`int j = ii.intValue() * 7;`

Java 1.5 - Autoboxing

- Java 1.5 introduce il meccanismo di autoboxing che consente conversioni fra classe wrapper e tipo associato in modo automatico:

- **Prima:**

```
Integer [] x = new Integer[10];  
x[0] = new Integer(42);  
int total = x[0].intValue();
```

- **Java 1.5:**

```
Integer [] x = new Integer[10];  
x[0] = 42;  
int total = x[0];
```